



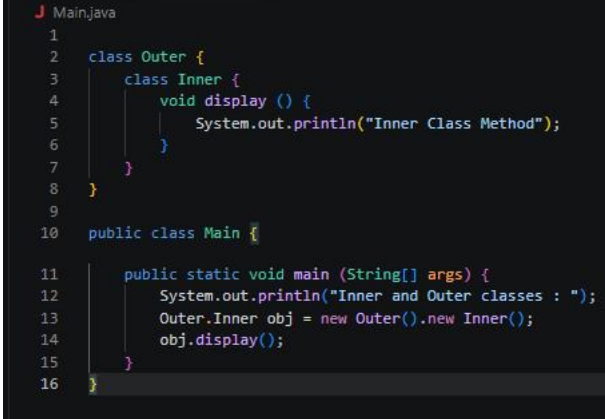
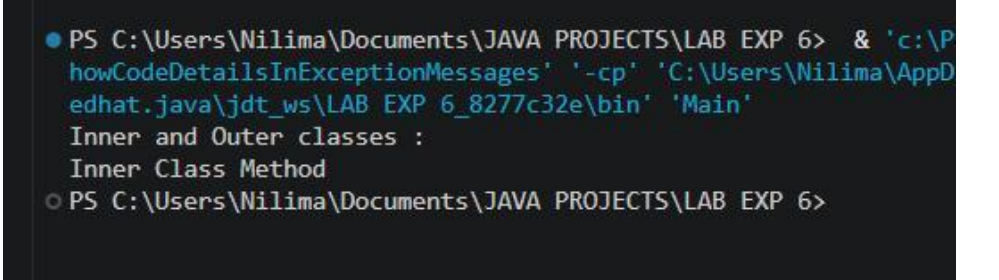
Symbiosis Institute of Technology, Pune

CSE- Academic Year 2026-27

Java Programming – Lab Batch 2025-29

Lab Assignment No.: 6

Name of Student	Akshit Gawade
PRN No.	25070122029
Batch	2025-2029
Class	A1
Academic Year & Semester	Second Year Semester III
Date of Submission	11/7/2026
Title of Assignment:	Write a Java program to demonstrate the use of an inner class, an anonymous class.
Problem Statement	Write a Java program to demonstrate the use of an inner class, an anonymous class.
Learning objectives	To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.
Theory	An inner class is a class defined inside another class and has access to all members of the enclosing class, including private members. Inner classes help improve code organization and logical grouping of related classes. Anonymous classes are unnamed inner classes that are instantiated only once and are mainly used for implementing interfaces or abstract classes. They simplify code by eliminating the need for separate class definitions. These concepts are widely used in Java GUI programming and event handling.

Program Code	 <pre> 1 Main.java 2 class Outer { 3 class Inner { 4 void display () { 5 System.out.println("Inner Class Method"); 6 } 7 } 8 } 9 10 public class Main { 11 12 public static void main (String[] args) { 13 System.out.println("Inner and Outer classes : "); 14 Outer.Inner obj = new Outer().new Inner(); 15 obj.display(); 16 } 17 } </pre>
Output Screenshots	 <pre> PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 6> & 'C:\P howCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Nilima\AppData edhat.java\jdt_ws\LAB EXP 6_8277c32e\bin' 'Main' Inner and Outer classes : Inner Class Method PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 6> </pre>
Exercise Qns	Q1. Create a Vehicle program where an inner class displays vehicle details and anonymous class performs an action.

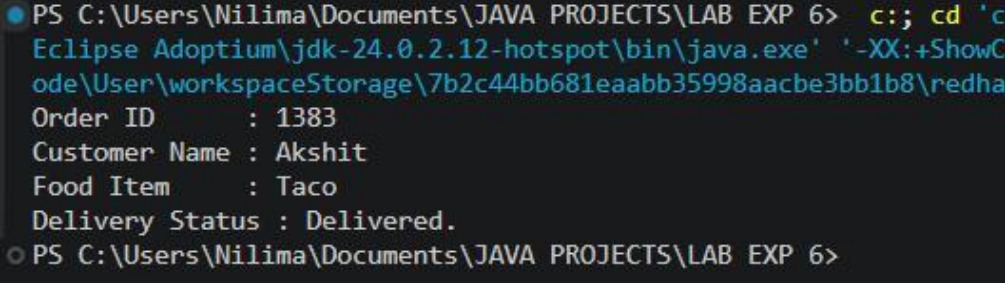
```
1
2 interface Action {
3     void start();
4 }
5
6 class Vehicle {
7     String name;
8     String model;
9
10    Vehicle (String n, String m) {
11        this.name = n;
12        this.model = m;
13    }
14
15    class Display {
16        void display() {
17            System.out.println("Car Name : " + name);
18            System.out.println("Car Model : " + model);
19        }
20    }
21 }
22
23 public class Ex1 {
24     Run | Debug
25     public static void main (String[] args) {
26         Vehicle.Display v = new Vehicle(n: "Kia", m: "Carnival").new Display();
27
28         Action a = new Action () {
29             @Override
30             public void start () {
31                 System.out.println(x: "Car engine started.");
32             }
33         };
34
35         v.display();
36         a.start();
37     }
38 }
```

Q2. Develop a Food Delivery Application where an inner class handles order details and anonymous classes handle delivery status updates.

```
1
2  interface DeliveryStatus {
3      void updateStatus();
4  }
5
6  class FoodDelivery {
7      int orderId;
8      String customerName;
9      String foodItem;
10
11      FoodDelivery (int orderId, String customerName, String foodItem) {
12          this.orderId = orderId;
13          this.customerName = customerName;
14          this.foodItem = foodItem;
15      }
16
17      class OrderDetails {
18          void display() {
19              System.out.println("Order ID      : " + orderId);
20              System.out.println("Customer Name : " + customerName);
21              System.out.println("Food Item    : " + foodItem);
22          }
23      }
24  }
25
26  public class Ex2 {
27      Run | Debug
28      public static void main(String[] args) {
29          FoodDelivery order = new FoodDelivery(orderId:1383, customerName:"Akshit", foodItem:"Taco");
30
31          FoodDelivery.OrderDetails details = order.new OrderDetails();
32          details.display();
33
34          DeliveryStatus status = new DeliveryStatus() {
35              @Override
36              public void updateStatus() {
37                  System.out.println(x: "Delivery Status : Delivered.");
38              }
39          };
40
41          status.updateStatus();
42      }
43  }
```

**Output
Screenshots**

```
Car started!
● PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 6> c:; cd 'c:\U
Eclipse Adoptium\jdk-24.0.2.12-hotspot\bin\java.exe' '-XX:+ShowCode
ode\User\workspaceStorage\7b2c44bb681eaabb35998aacbe3bb1b8\redhat.j
Car Name : Kia
Car Model : Carnival
Car engine started.
○ PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 6>
```

	 <pre> PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 6> c:; cd 'c:\Eclipse Adoptium\jdk-24.0.2.12-hotspot\bin\java.exe' -XX:+ShowCode\user\workspaceStorage\7b2c44bb681eaabb35998aacbe3bb1b8\redha Order ID : 1383 Customer Name : Akshit Food Item : Taco Delivery Status : Delivered. PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 6> </pre>
Conclusion	This assignment helped in understanding the concepts of inner classes and anonymous classes in Java. It demonstrated how an inner class can access the members of its enclosing class to improve code organization and encapsulation, while an anonymous class can provide a one-time implementation of an interface or abstract class without creating a separate class. The implementation improved my understanding of object-oriented programming concepts, event handling, and code reusability. These concepts are widely used in Java GUI development, event-driven programming, multithreading, and real-world applications where concise and modular code is required.